FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	Alvin Christian S. Panio
Section:	AN43-BSCSSE
Professor:	Crisola Tan
Date:	October 6, 2026

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
   (Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

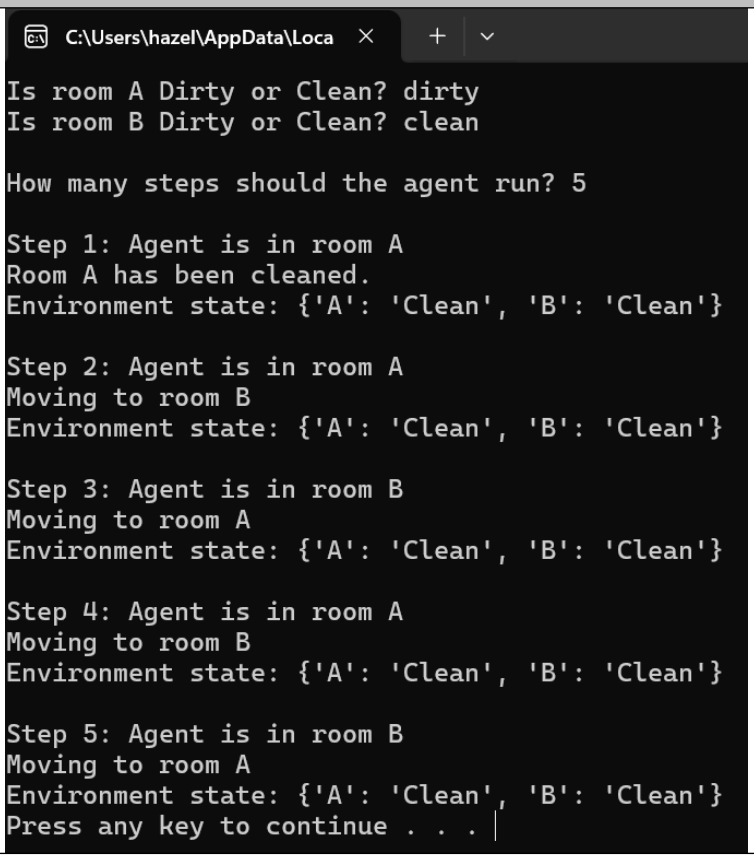
Pre-Lab Preparation

1. Review the Python concepts:
    - Classes and objects
    - If-Else statements
    - Loops (for loop)
    - Functions / methods
2. Understand the environment:
    - Two rooms: A and B
    - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output



Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.

2. Implement the **perceive-and-act method**:
    - Rule 1: If the current room is dirty, clean it.
    - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

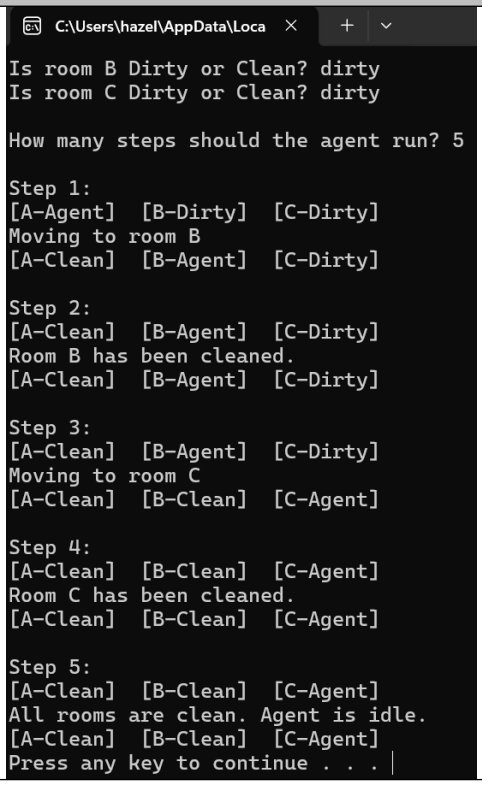
Task 3: Run the Simulation

1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.
3. After each step, print:
    - The agent's current location
    - The action taken (cleaned/moved)
    - Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)



Your Code - Laboratory Tasks ⇩

Completed Python Code

```
"""TA2 - Rule-Based Vacuum Cleaner Agent."""


def ask_room_state(room):
    """Ask for a valid initial state and return either Dirty or Clean."""
    while True:
        state = input(f"Enter the state of room {room} (Dirty/Clean): ").strip().lower()

        if state in ("dirty", "clean"):
            return state.title()

        print("Invalid input. Please enter only Dirty or Clean.")


def ask_number_of_steps():
    """Ask for a positive whole number of simulation steps."""
    while True:
        try:
            steps = int(input("How many steps should the simulation run? "))

            if steps > 0:
                return steps
        except ValueError:
            pass

        print("Invalid input. Please enter a positive whole number.")


def is_dirty(environment, room):
    """Return True when the selected room is dirty."""
    return environment[room] == "Dirty"


def clean_room(environment, room):
    """Change the selected room's state to Clean."""
    environment[room] = "Clean"


class RuleBasedAgent:
    """A vacuum agent controlled by two condition-action rules."""

    def __init__(self, environment, starting_room="A"):
        self.environment = environment
        self.location = starting_room

    def move(self):
        """Switch the agent between rooms A and B."""
        previous_room = self.location
        self.location = "B" if self.location == "A" else "A"
        return previous_room, self.location

    def perceive_and_act(self):
        """Observe the current room, apply one rule, and return the action."""
        # Rule 1: Clean the current room when it is dirty.
        if is_dirty(self.environment, self.location):
            cleaned_room = self.location
            clean_room(self.environment, cleaned_room)
            return f"Cleaned room {cleaned_room}."
```

```
        # Rule 2: Move when the current room is already clean.
        previous_room, new_room = self.move()
        return f"Moved from room {previous_room} to room {new_room}."


def main():
    """Set up the environment and run the requested simulation."""
    print("RULE-BASED VACUUM CLEANER AGENT")
    print("Set the initial state of each room.\n")

    # Task 1: Store the two-room environment in a dictionary.
    environment = {
        "A": ask_room_state("A"),
        "B": ask_room_state("B"),
    }

    # Task 2: Create the rule-based agent in room A.
    agent = RuleBasedAgent(environment, starting_room="A")
    steps = ask_number_of_steps()

    print("\nSIMULATION START")

    # Task 3: Run one perceive-and-act cycle during each step.
    for step in range(1, steps + 1):
        action = agent.perceive_and_act()

        print(f"\nStep {step}")
        print(f"Current location: Room {agent.location}")
        print(f"Action: {action}")
        print(f"Environment: {environment}")

    print("\nSIMULATION COMPLETE")


if __name__ == "__main__":
    main()
```

Simulation Output

```
RULE-BASED VACUUM CLEANER AGENT
Set the initial state of each room.

Enter the state of room A (Dirty/Clean): dirty
Enter the state of room B (Dirty/Clean): clean
How many steps should the simulation run? 5

SIMULATION START

Step 1
Current location: Room A
Action: Cleaned room A.
Environment: {'A': 'Clean', 'B': 'Clean'}

Step 2
Current location: Room B
Action: Moved from room A to room B.
Environment: {'A': 'Clean', 'B': 'Clean'}

Step 3
Current location: Room A
Action: Moved from room B to room A.
Environment: {'A': 'Clean', 'B': 'Clean'}

Step 4
Current location: Room B
Action: Moved from room A to room B.
Environment: {'A': 'Clean', 'B': 'Clean'}

Step 5
Current location: Room A
Action: Moved from room B to room A.
Environment: {'A': 'Clean', 'B': 'Clean'}
SIMULATION COMPLETE
```

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

Answer the Questions ⇩

1. The agent follows two rules: if the current room is Dirty, it cleans that room; if the current room is Clean, it moves to the other room.

2. The agent checks the state of its current room in the environment dictionary. It calls the cleaning function for Dirty and the move method for Clean.

3. If both rooms start Clean, the agent does not clean anything. It alternates between rooms A and B for the requested number of steps while both states remain Clean.

4. I would store the room names in a list or dictionary and track the current-room index. The next room can be selected with (current_index + 1) % number_of_rooms, allowing the same rules to work with any number of rooms.